Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

AI Job Helper

Project Engineering

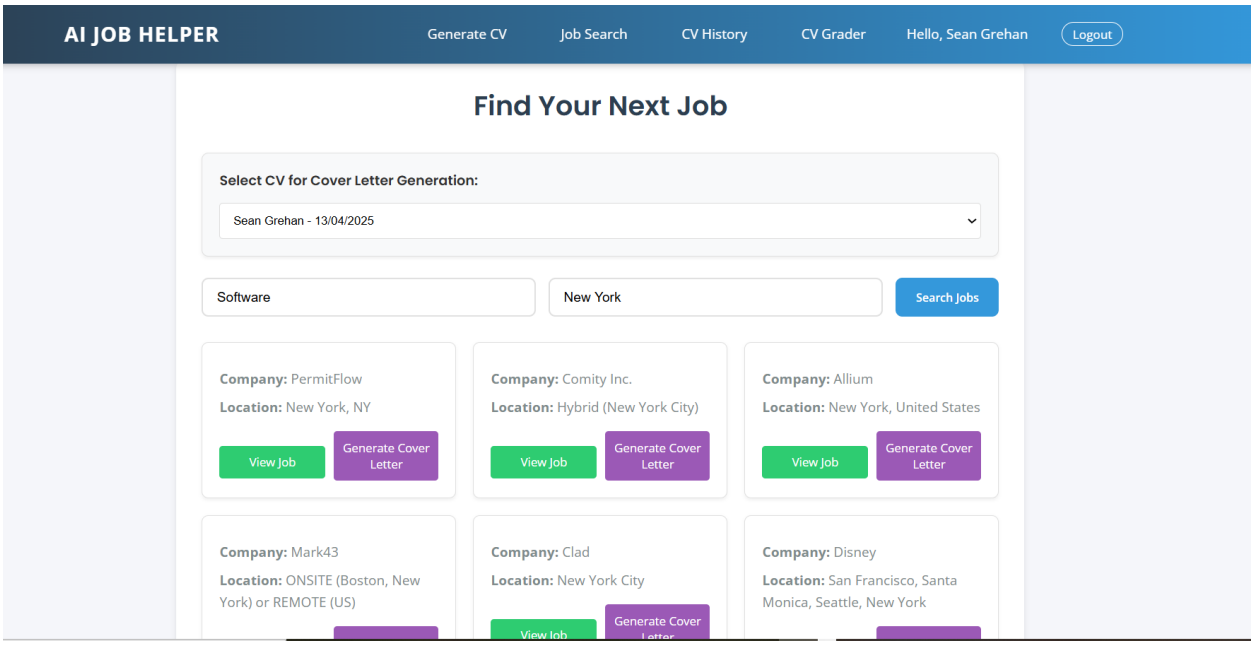
Year 4

Sean Grehan

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2024/2025


AI JOB HELPER
Generate CV
Job Search
CV History
CV Grader
Hello, Sean Grehan
Logout
Find Your Next Job
Select CV for Cover Letter Generation:
Sean Grehan - 13/04/2025
Software
New York
Search Jobs
Company: PermitFlow
Location: New York, NY
View Job
Generate Cover Letter
Company: Comity Inc.
Location: Hybrid (New York City)
View Job
Generate Cover Letter
Company: Allium
Location: New York, United States
View Job
Generate Cover Letter
Company: Mark43
Location: ONSITE (Boston, New York) or REMOTE (US)
Company: Clad
Location: New York City
Company: Disney
Location: San Francisco, Santa Monica, Seattle, New York

CV Evaluation Results

91
/100

A+

Evaluated for: **Software Engineer**

Section Breakdown

Education

11/15

The CV education section is clear and concise, presenting the degree, institution, and expected graduation date. It is well-formatted and consistent. However, it lacks detail about relevant coursework, projects, or academic achievements related to the Software Engineer position. Also, there are no mentions of relevant certifications or continuing education.

Work Experience

27/30

The CV experience section is quite strong overall. It starts each bullet point with a strong action verb, emphasizes achievements over responsibilities, quantifies achievements, and highlights experience relevant to the Software Engineer role. The candidate also demonstrates good use of past tense and focuses on recent and relevant experience. However, there is room for improvement in demonstrating progression and growth, and the level of detail could be slightly more specific.

Skills & Competencies

11/15

The CV skills section is generally well-structured and includes a variety of relevant skills for a Software Engineer role. The candidate demonstrates a good mix of technical skills, including proficiency in several programming languages and tools, as well as domain-specific skills such as cloud computing. However, the section lacks soft skills, which are also important in the software engineering field. The use of keywords relevant to ATS is good, but there is room for improvement. The CV avoids excessive jargon and acronyms without context.

Professional Summary

13/15

The professional summary is clear and concise, effectively articulating the candidate's career goals and professional identity as an aspiring software engineer. It highlights relevant skills and experience, such as proficiency in C, C++, Java, and Python, as well as an ongoing AWS certification. The summary also mentions a previous tenure at Microsoft, indicating experience in a professional setting. However, it lacks quantifiable achievements and could be more tailored to the target role.

Format & Presentation

9/10

The CV is well-structured, with clear and consistent headings and sections. The use of white space is effective, making the document easy to read and scan. The layout highlights key information such as the candidate's technical skills, education, and professional experience. The font choices and sizing are professional and consistent throughout the document. The organization of information is logical, starting with a professional summary, followed

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

_____

Acknowledgements

I would like to acknowledge all my family and friends who supported during the development if this project. I would like to thank all the ATU staff for their continued help and guidance throughout my time on the course. I want to thank all lecturers who helped with the project, no matter the impact, it's all appreciated. I want to thank my project supervisor Brian O'Shea for the support and teachings that helped to make this project come together. I lastly want to thank all my fellow classmates, who I became very close with over the last few years, I wish them all the best in their futures.

Contents

1 Summary 10

2 Poster 12

3 Introduction 13

4 Background ..... **Error! Bookmark not defined.**

5 Project Architecture 15

6 Project Plan ..... **Error! Bookmark not defined.**

7 React 16

7.1 React Component Architecture 16

7.2 State Management in React 16

7.3 React Hooks 18

8 Next.js 19

8.1 Next.js API Routes 20

8.2 Environment Variables and Configuration 23

8.3 Server-Side Database Operations 24

8.4 Advanced API Integration Patterns 26

9 OpenAI ChatGPT API 28

9.1 Core Use Cases 28

9.1.1 CV Generation 28

9.1.2 CV Improvement 29

9.1.3 CV Grading 30

9.1.4 Cover Letter Generation 31

9.2 Token Optimisation 31

9.2.1 Key Token Optimisation Strategies 32

9.2.2 Precision-Targeted Prompts 34
9.2.3 Token Limit Configuration 35
9.3 Prompt Engineering 36
9.3.1 Core Uses in the Project 37
9.3.2 Feature-Specific Prompt Engineering 46
9.3.3 Business Impact of Effective Prompt Engineering 52
10 FindWork API 53
10.1 Integration Architecture 53
10.2 Frontend Integration 55
10.3 Integration with CV Features 57
10.4 Security 59
10.5 Benefit of FindWork API 60
11 Users (Authentication and Session Management) 61
11.1 Core Technical Components 62
11.1.1 User Data 62
11.1.2 Session Management with Iron Session 63
11.1.3 Security 65
12 MongoDB 66
13 Problem Solving 69
14 Ethics 70
15 Conclusion 71
16 Appendix ..... **Error! Bookmark not defined.**
17 References 73

1 Summary

The purpose of this project is to help job seekers in creating high quality, Applicant Tracking System (ATS) avoidant CVs that best showcase their qualifications and unique abilities. The CV Generation and Improvement/Grading System lets users create, evaluate, and enhance professional resumes using OpenAIs ChatGPT Models.

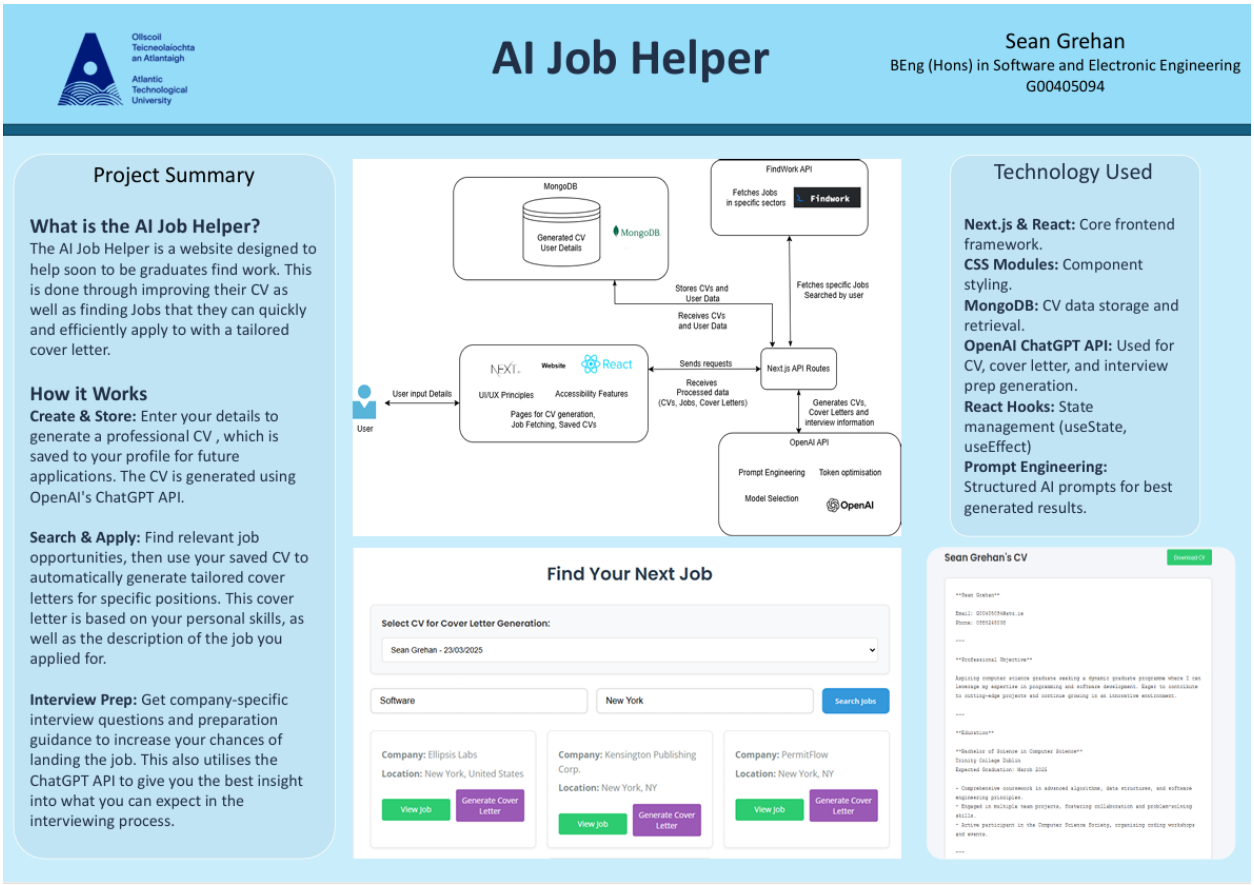
The scope of the project is a full-stack web application with user authentication, CV creation, CV grading, CV Improvement, Job Finding and applying, personalised cover letter generation and document management.

The project uses a systematic workflow: users create or upload CVs, receive a grade across multiple criteria (summary, achievements, relevant skills presentation, education formatting, and overall document structure), then generate improved versions of their CV.

To make this project I used React.js,Next.js, Node.js, next-iron-session, Fetch API, MongoDB and Mongoose, ChatGPT API and findWork API. I will expand on all of these in my introduction and how they work throughout the project.

The project is a perfect tool for job seekers and transforms average CVs into professional documents that can pass automatic Applicant Tracking Systems (ATS) that reject potential employees.

2 Poster


Oilscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University
AI Job Helper
Sean Grehan
BEng (Hons) in Software and Electronic Engineering
G00405094
Project Summary
What is the AI Job Helper?
The AI Job Helper is a website designed to help soon to be graduates find work. This is done through improving their CV as well as finding Jobs that they can quickly and efficiently apply to with a tailored cover letter.
How it Works
Create & Store: Enter your details to generate a professional CV , which is saved to your profile for future applications. The CV is generated using OpenAI's ChatGPT API.
Search & Apply: Find relevant job opportunities, then use your saved CV to automatically generate tailored cover letters for specific positions. This cover letter is based on your personal skills, as well as the description of the job you applied for.
Interview Prep: Get company-specific interview questions and preparation guidance to increase your chances of landing the job. This also utilises the ChatGPT API to give you the best insight into what you can expect in the interviewing process.
MongoDB
Generated CV User Details
MongoDB
FindWork API
Fetches Jobs in specific sectors
Findwork
Stores CVs and User Data
Receives CVs and User Data
Fetches specific Jobs Searched by user
User
User input Details
Website
React
UI/UX Principles
Accessibility Features
Pages for CV generation, Job Fetching, Saved CVs
Sends requests
Receives Processed data (CVs, Jobs, Cover Letters)
Next.js API Routes
Generates CVs, Cover Letters and interview information
OpenAI API
Prompt Engineering
Token optimisation
Model Selection
OpenAI
Find Your Next Job
Select CV for Cover Letter Generation:
Sean Grehan - 23/03/2025
Software
New York
Search Jobs
Company: Ellipsis Labs
Location: New York, United States
View Job
Generate Cover Letter
Company: Kensington Publishing Corp.
Location: New York, NY
View Job
Generate Cover Letter
Company: PermitFlow
Location: New York, NY
View Job
Generate Cover Letter
Technology Used
Next.js & React: Core frontend framework.
CSS Modules: Component styling.
MongoDB: CV data storage and retrieval.
OpenAI ChatGPT API: Used for CV, cover letter, and interview prep generation.
React Hooks: State management (useState, useEffect)
Prompt Engineering: Structured AI prompts for best generated results.
Sean Grehan's CV
Download CV

3 Introduction

As a student in the final year of my course, one of the main worries for me and any other student is our career after college. Many students I know now are faced with a tough job market which is making it more difficult to find a job than previous years. With the increased competitive nature of job seeking, applicants are looking for any help or tool that can assist them. Many current job seekers are turning to AI chat bots like ChatGPT or DeepSeek to try to generate CVs for them from scratch. The problem with this is that these CV's are easily identified as AI generated and don't truly reflect the applicant and their skills. An employer who see's a completely generated CV will be put off by this, hurting the applicant's chances of getting hired. Knowing this, ironically means that in trying to improve their CV, the applicant has actually decreased their chances of being hired. However, many employers working in HR report that most resumes received in from applicants use some form of AI generation to enhance their application and go as far to say that "if you aren't using AI for help, then you are falling behind". This is where my project comes into place. My project is designed around the concept of enhancing CVs and making them as strong as possible with the help of AI, as well as streamlining the job application process. My project aims to help all job seekers, of any technical level, achieve strong and enhanced CV's with the help of AI, so that they don't fall behind in the current job market.

The core of the project is built using: **React.js**- the JavaScript library used for building the user interface, **Next.js-** A popular React framework used for building the application It handles routing (both frontend pages and backend API routes), server-side rendering, and provides a structured way to organize the project. **Node.js-** The JavaScript runtime environment used to execute the backend (API routes) code on the server.

The frontend uses **Fetch API**- Standard browser/Node.js API used by frontend JavaScript to make requests to the backend API routes (e.g., submitting the form data, requesting CV grading).

The backend uses **Next.js API Routes**- The mechanism used to create server-side endpoints within Next.js. **next-iron-session:** A library used for managing user sessions securely using encrypted cookies. Used for authentication and ensuring users can only access their own data.

Fetch API- Used by the backend API routes to make requests to external services, specifically the OpenAI API.

MongoDB and **Mongoose** are used for the database. **MongoDB:** A NoSQL document database used to store application data, primarily the user-generated CVs (cvs collection). CRUD functions are used to change and update the data in the database. **Mongoose**: An Object Data Modeling (ODM) library for MongoDB and Node.js. It simplifies database interactions by providing schemas (models/CV.js), models, and methods (.findById, .findOne, .create, .findOneAndUpdate, .findOneAndDelete, etc.) to work with MongoDB data.

The two main external APIs that bring this project together are OpenAI's ChatGPT API and FindWork's dev API. The ChatGPT API is used for all generations(CVs, CoverLetters etc.) and also the grading system/algorithm. I have access to multiple of their models(gpt-4o, gpt-3.5 turbo etc.) with more continuing to come out often which are available to developers, meaning with time the project can easily be scaled to use modern models as they come out. This means that as AI models improve, so will the generations outputted from my app. The findWork API is used to fetch jobs, they have a large database of jobs they have collected, and their free API lets me fetch them and display them on the website. As I'll explain later, the API's work together for the Cover Letter generation.

4 ATS Keywords

Applicant Tracking Systems (ATS) is used widely by companies to streamline the hiring process, and using the right keywords is crucial for job seekers. ATS keywords are specific words or phrases that recruiters program into these systems to identify qualified candidates. If a resume doesn't include enough of these important terms, it may never reach a human recruiter at all. Companies rely on ATS to sift through hundreds or thousands of applications, using automatic detectors to weed out those that don't match the job description closely enough [1].Sometimes the user is completely qualifies, but don't happen to have the correct keywords, so they miss out on the opportunity. This project intends to combat that so people have a fair chance to get hired.

5 Project Architecture

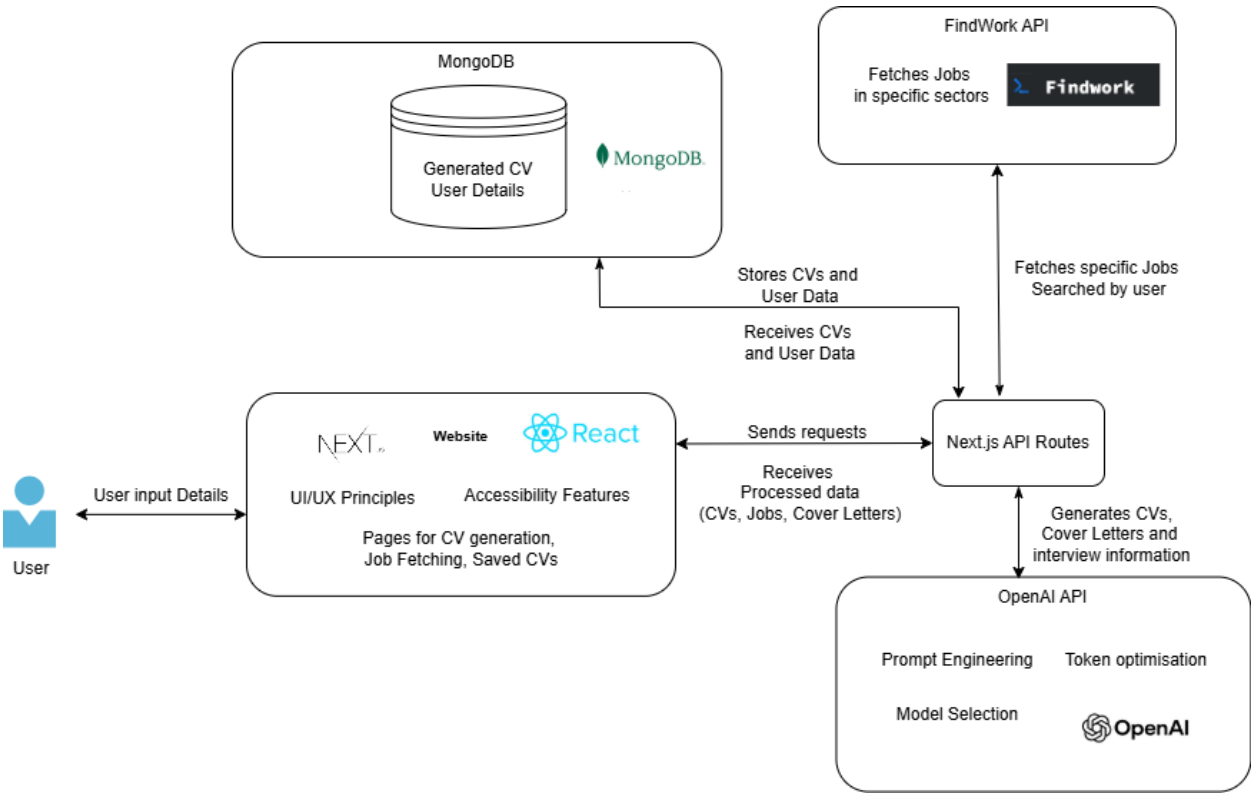


Figure 5-1 Architecture Diagram

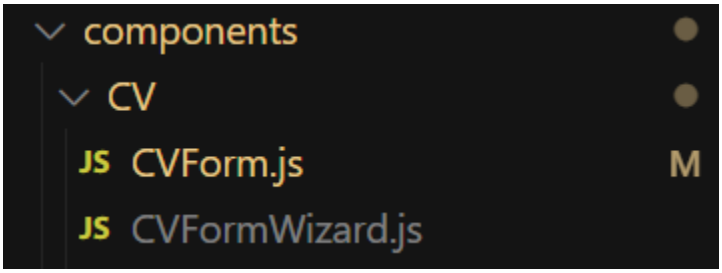
6 React

React is a JavaScript library developed by Facebook (Meta) for building user interfaces(UI). React allows developers to create reusable UI components that update when data changes. Instead of manipulating the DOM directly (which can be slow), React uses a "Virtual DOM" which is an in-memory representation of the real DOM, this means that only what is changed gets re-rendered.

In my project, React serves as the foundation for the entire frontend, providing the interactive user experience where users can input their information, generate CVs, request improvements, and view results.

6.1 React Component Architecture

In this project, the frontend is organised into a hierarchy of React components which are responsible for specific parts of the UI. Components like CVForm and CVFormWizard contain the functionality of specific features in the app. The CVForm.js component handles the main CV creation interface, while CVFormWizard.js breaks this process into manageable steps for users.



The component architecture follows the principle of "composition over inheritance," with smaller, focused components combined to create more complex UIs.

6.2 State Management in React

State management is the most crucial aspect of React in this application. The CV builder manages complex state that includes:

User input data (personal details, experience, education, skills)

Application status (loading states, success/error messages, current view)

Generated content

UI preferences (selected templates)

When state changes (e.g. a user types in a field or an API call completes), React automatically re-renders affected components to reflect the new state. This is what I mentioned earlier “only what is changed gets re-rendered.”

The useState hook is the primary tool in this application:

```
// From components/CV/CVForm.js
const [formData, setFormData] = useState({
  name: '',
  email: '',
  phone: '',
  linkedin: '',
  objective: '',
  skills: '',
  experience: [{ position: '', company: '', duration: '', responsibilities: '' }],
  education: [{ institution: '', degree: '', duration: '' }],
  certifications: '',
  hobbies: ''
});
const [isLoading, setIsLoading] = useState(false);
const [generatedCV, setGeneratedCV] = useState('');
const [error, setError] = useState('');
const [savedCVId, setSavedCVId] = useState(null);
const [selectedTemplate, setSelectedTemplate] = useState('professional');
```

This code demonstrates React's principle of "atomic state" which is breaking down application state into focused pieces that serve specific purposes:

formData holds all the user's CV information in a structured object

isLoading tracks when API requests are in progress to display loading indicators

generatedCV stores the HTML content of the CV after generation. This is used for allowing the user to download the word document.

error maintains any error messages that need to be displayed

savedCVId keeps the database ID of the saved CV for reference

selectedTemplate tracks which CV template design the user has chosen

Each state variable comes with its own updater function (e.g. setFormData) that provides a controlled way to modify state.

6.3 React Hooks

```
// From components/CV/CVForm.js
// State hooks for managing data
const [formData, setFormData] = useState({/* initial data */});

// Effect hook for side effects like fetching data
useEffect(() => {
  // This effect runs after render and when dependencies change
  if (savedCVId) {
    // Fetch additional details or update UI based on saved status
    document.title = `CV Created - ID: ${savedCVId}`;

    // Could also trigger analytics events, notifications, etc.
    analytics.trackEvent('cv_generated', { template: selectedTemplate });
  }

  // Optional cleanup function runs before the component unmounts
  // or before the effect runs again
  return () => {
    // Clean up any subscriptions, timers, etc.
  };
}, [savedCVId, selectedTemplate]); // Dependencies - effect runs when these values change
```

```
// Event handler using state updater
const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prevData => ({
    ...prevData,  // Preserve existing form data
    [name]: value // Update only the changed field
  }));
};
```

This code demonstrates three fundamental aspects of React hooks:

1. **State Hook (useState)**:

- Creates a state variable (formData) and its updater function (setFormData), like I talked about in the State Management section.

- Enables the component to "remember" information between renders
- When the updater is called, it triggers a re-render with the new value

2. **Effect Hook (useEffect)**:

- Executes side effects after render
- The function runs after initial render and whenever dependencies change
- In this example, it updates the document title and tracks when a CV is saved
- The dependency array [savedCVId, selectedTemplate] controls when the effect reruns
- The optional cleanup function helps prevent memory leaks

3. **State Updates with Previous State**:

- The handleInputChange function uses a functional update pattern
- By passing a function to setFormData that receives the previous state value
- This ensures accurate updates even if multiple state updates are batched together

Hooks transform the component from a static template into a dynamic, reactive system that responds to user input, API responses, etc..

7 Next.js

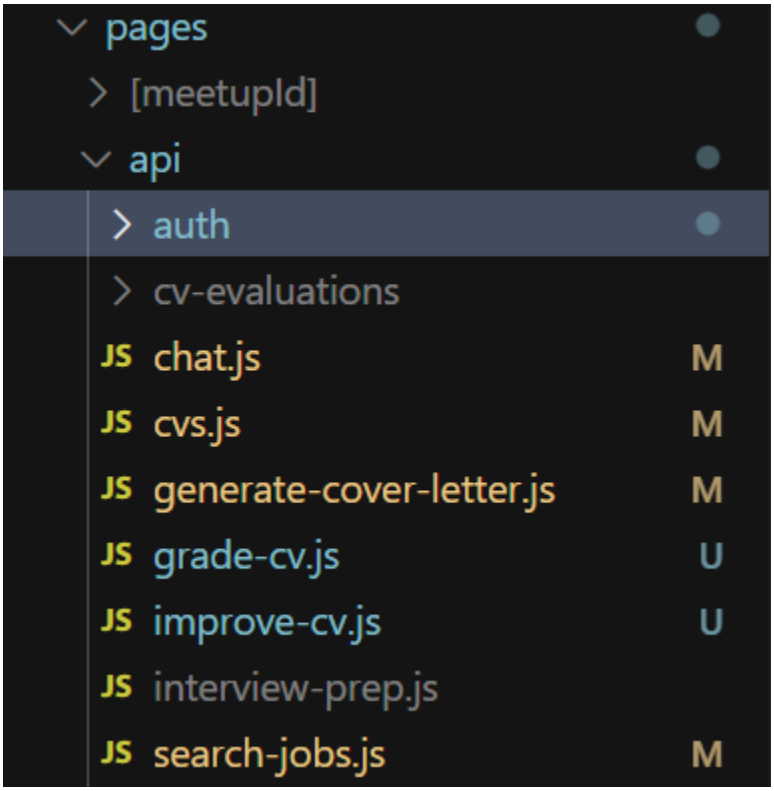
Next.js is a powerful React framework created by Vercel that improves React's capabilities with server-side functionality, routing and built in API routes. React focuses on UI components, but Next.js is necessary to build complete applications[3].

In the AI Job Helper, Next.js is used for both the frontend page structure and the backend API endpoints that power the application's core features. It bridges the gap between client and server, integrating the React components with server-side processing and external API calls[2].

7.1 Next.js API Routes

The most prominent Next.js feature in this project is API Routes, which allow creating serverless backend endpoints within the application. By placing my JavaScript files in the /pages/api directory, Next.js automatically creates corresponding API endpoints that can be called from the frontend. Here are some of the important API routes:

- /api/chat.js - Generates CV content using OpenAI
- /api/cvs.js - Manages saved CV documents
- /api/grade-cv.js - Analyses and scores CV content
- /api/improve-cv.js - CV improvements on graded CVs
- /api/generate-cover-letter.js - Creates matching cover letters



Let me explain further using this code snippet from the project(next three screenshots):

```
// From pages/api/chat.js
import dbConnect from '../../lib/mongodb';
import CV from '../../models/CV';
import { withSessionRoute, getUserFromSession } from '../../lib/session';

/**
 * CV Generation API Route
 * This API endpoint processes user-submitted CV form data, formats it for OpenAI,
 * sends a request to the OpenAI API to generate a professional CV, and stores
 * both the original data and the generated CV in MongoDB.
 */
async function handler(req, res) {
  // Only allow POST requests
  if (req.method !== 'POST') {
    res.setHeader('Allow', ['POST']);
    return res.status(405).json({ message: `Method ${req.method} Not Allowed` });
  }

  try {
    // Check if user is authenticated
    const user = await getUserFromSession(req);
    if (!user) {
      return res.status(401).json({ message: 'Authentication required' });
    }
```

```
  try {
    // Check if user is authenticated
    const user = await getUserFromSession(req);
    if (!user) {
      return res.status(401).json({ message: 'Authentication required' });
    }

    // Connect to the database
    await dbConnect();

    // Extract form data and template selection from request body
    const { formData, template } = req.body;

    // Call OpenAI API to generate CV content
    // ...OpenAI API call code...

    // Save to database
    const newCV = new CV({
      formData: formData,
      generatedCV: formattedContent,
      rawGeneratedCV: rawContent,
      template: template,
      userId: user.id,
      documentTitle: documentTitle,
    });
```

```
      documentTitle: documentTitle,
    });

    const savedCV = await newCV.save();

    // Return success response with generated content
    return res.status(200).json({
      message: 'CV generated and saved successfully!',
      response: formattedContent,
      savedCV: savedCV._id
    });

  } catch (error) {
    console.error('API Route /api/chat Error:', error);
    return res.status(500).json({ message: 'An internal server error occurred while processing th
  }
}

// Export the handler wrapped with session management
export default withSessionRoute(handler);
```

This code shows:

1. **File Based Routing**:

- The file at pages/api/chat.js automatically becomes available at the /api/chat endpoint
- No routing configuration needed

2. **Request and Response Handling**:

- The handler function receives req (request) and res (response) objects
- req.method can be used to check the HTTP method (GET, POST, etc.)
- req.body contains the parsed JSON body of POST requests
- res.status() and res.json() methods send responses back to the client

3. **Server-Side**:

- API routes run exclusively on the server, never in the browser
- They have access to server side resources like the MongoDB database, file systems, and environment variables where my API keys are stored
- They can use Node.js modules not available in the browser

4. **Middleware Integration**:

- API routes can be wrapped with middleware like withSessionRoute[2]
- This enables shared functionality across routes, used for authentication, logging, etc.[3]

5. **Error Handling**:

- Try/catch blocks manage errors during processing
- Appropriate HTTP status codes (401, 404, 500, etc.) communicate errors to clients

The API route structure allows the project to separate business logic from the React components, making a cleaner architecture with separation of concerns.

7.2 Environment Variables and Configuration

Next.js lets me manage environment variables, which is great for handling sensitive information like my API keys and database credentials.

I'll show how the environment variables are accessed across the application:

```
// From pages/api/grade-cv.js - Using environment variables for API keys
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}` // Environment variable access
  },
  body: JSON.stringify({
    model: 'gpt-4o',
    messages: [{ role: 'user', content: prompt }],
    temperature: 0.3,
    max_tokens: max_tokens,
    response_format: { type: "json_object" }
  }),
  signal: controller.signal, // For timeout handling
});
```

1. **Environment Variable Access**:

- process.env.OPENAI_API_KEY gives access to environment variables

- In API routes, this accesses the server side environment variables

2. **Security Benefits**:

- API keys like OPENAI_API_KEY never appear in the source code
- They're never exposed to the browser
- I define my values(API keys and Database connection) in .env.local for local development
- I can securely upload my code to GitHub without the risk of my API keys being publicly available as my .env file won't be pushed to the repository.

7.3 Server-Side Database Operations

Next.js API routes provide a server environment where the application can safely connect to databases[2]. In the project, this is used to store and retrieve user CV data from MongoDB using the Mongoose ODM.

```
// From pages/api/cvs.js - Database operations in Next.js API routes
import dbConnect from '../../lib/mongodb';
import CV from '../../models/CV';
import { withSessionRoute, getUserFromSession } from '../../lib/session';

async function handler(req, res) {
  const { method } = req;

  // Connect to the database
  await dbConnect();

  // Get the logged in user
  const user = await getUserFromSession(req);

  // Check if user is authenticated
  if (!user) {
    return res.status(401).json({ success: false, error: 'Not authenticated' });
  }

  switch (method) {
    case 'GET':
      try {
```

```
switch (method) {
  case 'GET':
    try {
      // If an ID is provided, get a specific CV
      if (req.query.id) {
        const cv = await CV.findOne({
          _id: req.query.id,
          userId: user.id // Security: Only return CV if it belongs to the logged in user
        });

        if (!cv) {
          return res.status(404).json({ success: false, error: 'CV not found' });
        }

        res.status(200).json({ success: true, data: cv });
      }
      // Otherwise, get all CVs for the current user
      else {
        const cvs = await CV.find({ userId: user.id });
        res.status(200).json({ success: true, data: cvs });
      }
    } catch (error) {
      res.status(400).json({ success: false, error: error.message });
    }
    break;
```

1. **Database Connection**:

- dbConnect() connects to MongoDB
- This utility from lib/mongodb.js handles connection pooling and configuration[2]
- API routes can safely connect to the database without exposing credentials to clients

2. **Model Imports**:

- import CV from '../../models/CV' brings in the Mongoose model
- This model defines the schema and provides methods for the database

3. **CRUD Operations**:

- Create: CV.create(cvData)
- Read: CV.findOne() and CV.find() (shown above)
- Update: CV.findOneAndUpdate()
- Delete: CV.findOneAndDelete()

4. **Security Filtering**:

- Queries include userId: user.id to ensure users can only access their own data.
- Next.js's server side environment makes sure this filtering can't be bypassed by clients

5. **Response Formatting**:

- API routes format database results into JSON responses

This database integration pattern allows the CV builder to keep user data securely while still keeping database credentials and logic safely on the server.

7.4 Advanced API Integration Patterns

The project uses patterns for using the external APIs, especially OpenAI. Next.js API routes are a secure middleware layer, dealing with the complexity of these integrations from the frontend.

```
// From pages/api/improve-cv.js - Multi-step API integration pattern
// First do a quality check on the CV with a cheaper model
const qualityCheckResponse = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-3.5-turbo', // Use cheaper model for initial analysis
    messages: [
      {
        role: 'system',
        content: 'You are a CV analysis expert who identifies the key information in a CV.'
      },
      {
        role: 'user',
        content: `Analyze this CV and extract the key information (job titles, years of experienc
      }
    ],
    temperature: 0.3
  })
});
```

```
if (!qualityCheckResponse.ok) {
  return res.status(qualityCheckResponse.status).json({
    error: 'Failed to analyze CV',
  });
}

const qualityCheckData = await qualityCheckResponse.json();
const keyInformation = qualityCheckData.choices[0].message.content.trim();

// Make second request to OpenAI API with enhanced model and parameters
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4-turbo', // Use more powerful model for the main task
    messages: [
      {
        role: 'system',
        content: 'You are an elite CV optimization specialist who transforms average CVs into exc
      },
      {
        role: 'user',
        content: `Key information that MUST be preserved in the improved CV:\n${keyInformation}\n`
```

```
// Make second request to OpenAI API with enhanced model and parameters
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4-turbo', // Use more powerful model for the main task
    messages: [
      {
        role: 'system',
        content: 'You are an elite CV optimization specialist who transforms average CVs into exc
      },
      {
        role: 'user',
        content: `Key information that MUST be preserved in the improved CV:\n${keyInformation}\n`
      }
    ],
    temperature: 0.3,
    max_tokens: 4000,
    top_p: 0.9
  })
});
```

1. **Multi-Step Processing**:

- Uses a cheaper model (gpt-3.5-turbo) for to get the initial facts

- Feeds those facts into a more expensive model (gpt-4-turbo) for the main improvement task
- Each step builds on previous results

2. **API Key Protection**:

- The OpenAI API key is stored securely in environment variables(mentioned earlier)

3. **Error Handling and Status Propagation**:

- Checks qualityCheckResponse.ok to detect errors in the first API call

4. **Cost Optimisation**:

- Uses different models based on the task requirements
- Saves costs by using gpt-3.5-turbo for simpler analysis
- Uses the more expensive gpt-4-turbo for the more complex improvement task

5. **Request Parameters**:

- Configures each request with appropriate parameters (temperature, max_tokens, top_p), which are essential for token optimisation[2][4].

8 OpenAI ChatGPT API

The OpenAI ChatGPT API is a gives me access to their large language models (LLMs) like GPT-3.5 and GPT-4. These models can understand and generate human like text based on the prompts they receive, which is perfect for creating the CVs and cover letters. The OpenAI API serves as the core intelligence that powers most of the features: generating professional CV content, improvement suggestions, grading CVs, and creating cover letters.

8.1 Core Use Cases

8.1.1 CV Generation

The CV generation process:

1. **Data Collection**:

- User fills out the CV form with personal information, education, experience, etc.
- This data is sent from the frontend to the /api/chat API route

2. **Prompt Engineering**:

- Form data is transformed into a structured prompt. I'll go further into detail of the prompt engineering later in the report as it turned out to be a vital part to improving the overall quality of the project
- The prompt has the specific instructions for the AI to create a professional CV in the way I want
- Data is organised into sections that map to the CV components

3. **AI Request**:

- The prompt is sent to OpenAI with the specific parameters mentioned earlier:
- model: 'gpt-4o'
- temperature: 0.5 - Balancing creativity and consistency
- max_tokens: 2500 – Gives enough space for a complete CV

4. **Response Processing**:

- The generated CV text is extracted from the API response
- Formatted into the template the user chose
- Stored in the database and returned to the frontend

8.1.2 CV Improvement

The CV improvement process:

1. **Multi Stage Approach**:

- First uses model GPT-3.5 to extract the information

- Then uses GPT-4 for the actual improvement

2. **Detailed Instructions**:

- Provides specific guidance on what improvements to make
- Sets clear boundaries on what should and shouldn't be changed
- Includes both what to do and what not to do

8.1.3 CV Grading

The CV grading process:

1. **Structured Output**:

- Requests responses in a JSON structure
- Has a score for each section
- Gives feedback in categories (strengths, weaknesses, suggestions)

2. **System Prompt**:

- Makes the AI out as an expert CV reviewer and recruiter
- Asks for detailed analysis
- Want’s constructive and easy to take on feedback

3. **Assessment Areas**:

- Scores format, content, impact, relevance, and clarity

4. **Response Processing**:

- Uses response_format: { type: "json_object" } parameter to ensure valid JSON[2]
- Everything is parsed then sent to the front end

8.1.4 Cover Letter Generation

1. **Contextual Input**:

- Takes both the user's CV and a specific job description(from FindWork's API)
- Creates a personalised cover letter targeted to the specific position

2. **Detailed Instructions**:

- Gives guidelines for structure and content
- Asks for a proper business letter format, appropriate length and tone

3. **Emphasis**:

- Asks the AI to highlight the most relevant qualifications
- Needs to have connections between experience and job requirements
- Asks for specific examples rather than generic statements

4. **Professional Output**:

- Creates a consistent matching document to accompany the CV

8.2 Token Optimisation

Token optimisation refers to strategies for efficiently using OpenAI's API by reducing the number of tokens used while keeping good output quality. Tokens are the basic units of text that the API processes, and they directly impact both costs and performance[4]. In this project, token optimisation is needed because it:

1. Reduces API costs
2. Improves response times by processing smaller text chunks[4]
3. Avoids hitting token limits that could shorten/cutoff important outputs
4. Allows me to get the most use out of different model tiers

The AI Job Helper uses several strategies to optimise token usage while still delivering high-quality outputs.

8.2.1 Key Token Optimisation Strategies

Tiered Model Selection:

```
// From pages/api/improve-cv.js - Tiered model approach
// First analyze with cheaper model
const qualityCheckResponse = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-3.5-turbo', // Cheaper model for initial analysis
    messages: [
      {
        role: 'system',
        content: 'You are a CV analysis expert who identifies the key information in a CV.'
      },
      {
        role: 'user',
        content: `Analyze this CV and extract the key information (job titles, years of experienc
      }
    ],
    temperature: 0.3
  })
});
```

```
// Use more powerful model for final output
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4-turbo', // More expensive but higher quality model
    messages: [
      {
        role: 'system',
        content: 'You are an elite CV optimization specialist who transforms average CVs into exc
      },
      {
        role: 'user',
        content: `Key information that MUST be preserved in the improved CV:\n${keyInformation}\n
      }
    ],
    temperature: 0.3,
    max_tokens: 4000
  })
});
```

This strategy:

1. **Cost-Based Model Selection**:

- Uses GPT-3.5 Turbo (8x cheaper than GPT-4) for simpler extraction tasks
- Reserves GPT-4 for complex tasks that need advanced reasoning

2. **Task Complexity Matching**:

- Aligns model capabilities with task requirements
- GPT-3.5 Turbo is better for simple fact extraction
- GPT-4 is better for creative improvement and complex reasoning

3. **Sequential Processing Pipeline**:

- Initial cheaper model extracts key facts
- Passes only essential information to the more expensive model
- Reduces total tokens processed by the premium model

4. **Improved Token Efficiency**:

- May process 1000 tokens with GPT-3.5 to extract key points
- Feeds those key points into GPT-4
- Versus sending the entire 1000 tokens directly to GPT-4

The token cost difference is substantial: GPT-4 costs around $0.03 per 1K input tokens versus $0.0035 for GPT-3.5, making this tiered approach about 8 times more cost-efficient[4][2].

8.2.2 Precision-Targeted Prompts

```
  "overallScore": <number 1-10>,
  "sections": {
    "format": {
      "score": <number 1-10>,
      "feedback": "<brief specific feedback>"
    },
    "content": {
      "score": <number 1-10>,
      "feedback": "<brief specific feedback>"
    },
    "impact": {
      "score": <number 1-10>,
      "feedback": "<brief specific feedback>"
    },
    "clarity": {
      "score": <number 1-10>,
      "feedback": "<brief specific feedback>"
    }
  },
  "strengths": ["<strength1>", "<strength2>", "<strength3>"],
  "weaknesses": ["<weakness1>", "<weakness2>", "<weakness3>"],
  "suggestions": ["<suggestion1>", "<suggestion2>", "<suggestion3>"],
  "summary": "<1-2 sentence overall assessment>"
}

Keep all feedback brief and focused. Do not include ANY other text outside the JSON structure.`;
```

This approach optimizes tokens by:

1. **Eliminating generic language**:

- Uses concise, direct language without uneeded elaboration
- Tells the model to "keep all feedback brief and focused"
- Avoids wasting tokens on lengthy explanations

2. **Structured Output Requirements**:

- Defines exactly what information is needed
- Stops ambiguity that could lead to longer responses

3. **Response Constraints**:

- Explicitly asks for "brief specific feedback"
- Limits the number of items in lists (3 strengths, 3 weaknesses, 3 suggestions)
- Specifies "1-2 sentence overall assessment" for summary

4. **Clear Boundaries**:

- Instructs the model to focus "ONLY on these categories"
- Explicitly states "Do not include ANY other text outside the JSON structure"

These prompts can reduce output tokens by 30-50% compared to more open-ended prompts.

8.2.3 Token Limit Configuration

```
body: JSON.stringify({
  model: 'gpt-4o',
  messages: [
    {
      role: 'system',
      content: 'You are an expert CV writer who creates professional, impactful CVs.'
    },
    {
      role: 'user',
      content: cvPrompt // The formatted CV data and instructions
    }
  ],
  temperature: 0.5,
  max_tokens: 2500 // Carefully calibrated token limit
})
```

1. **Task-Appropriate Limits**:

- Sets 2500 tokens for CV generation (enough for a complete CV)
- Uses lower limits (e.g., 1000) for analysis tasks

- Increases limits (e.g., 4000) for improvement tasks that need more context

2. **Prevents Wasted Tokens**:

- Avoids unnecessarily high limits that might generate longer content than necessary
- Stops generation when sufficient content is produced
- Reduces cost by not allowing the model to ramble

3. **Balances Completeness and Efficiency**:

- Ensures enough tokens for high-quality, complete responses
- Avoids cutting off important content
- Optimises for the specific needs of each feature

By setting appropriate max_tokens values, the application avoids both:

- Setting limits too low (cutting off important content)
- Setting limits too high (wasting tokens on unnecessary content)

For example, setting a 4000 token limit when 2500 is sufficient could potentially waste 1500 tokens on every request, increasing costs by 60% with no benefit to the output quality.

8.3 Prompt Engineering

Prompt engineering refers to the crafting of effective instructions/prompts for AI language models to produce the best outputs. It involves designing precise, clear instructions that guide the model to generate responses that meet specific requirements in terms of format, style, tone, and content[4][2].

In the AI Job Helper, prompt engineering is crutial. The application needs to transform the users information into professional CV content, analyse their existing CVs, suggest improvements, and generate cover letters which are all tasks that require carefully constructed, unique prompts to get the best results.

The project uses prompt engineering techniques that let the AI function as an expert CV writer, reviewer, and career advisor, giving professional quality outputs consistently.

8.3.1 Core Uses in the Project

8.3.1.1 Role-Based Instruction

```
// From pages/api/grade-cv.js - Role-based instruction
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4o',
    messages: [
      {
        role: 'system',
        content: 'You are an expert CV reviewer with extensive experience in recruiting across mu
      },
      {
        role: 'user',
        content: `Thoroughly review the following CV and provide a detailed assessment with score
      }
    ],
    temperature: 0.3,
    max_tokens: 3000,
    response_format: { type: "json_object" }
  })
});
```

This technique:

1. **Establishes Expert Persona**:

- Creates a clear identity for the AI as an "expert CV reviewer"
- Defines behavioural characteristics (detailed, fair, actionable)
- Sets tone (professional, constructive)

2. **Separates Setup from Task**:

- System message configures the AI's approach and expertise
- User message contains the specific task and input data

- Creates cleaner, more focused prompts for each purpose

3. **Consistent Behaviour**:

- The persona stays the same across different user inputs
- Maintains professional tone regardless of CV quality

4. **Improves Response Quality**:

- Asks for expert level analysis by framing the AI as an expert
- Encourages 'actionable' feedback by making it part of the persona
- Sets clear expectations for the type of response wanted

The system/user message separation is good for keeping a consistent expert voice across the different features, making sure the users get professional quality assistance regardless of which feature they use.

8.3.1.2 Detailed Context and Background Information

```
// From pages/api/improve-cv.js - Detailed contextual prompt
const systemPrompt = `You are an elite CV optimization specialist who transforms average CVs into

When improving CVs, you follow these key principles:
1. Highlight measurable achievements rather than duties
2. Use powerful action verbs and industry-specific terminology
3. Quantify accomplishments wherever possible (numbers, percentages, etc.)
4. Tailor content to emphasize relevance to the target role/industry
5. Create clean, consistent formatting with proper section hierarchy
6. Eliminate generic clichés and vague statements
7. Ensure all information flows logically and tells a cohesive career story

YOUR TASK: Transform the provided CV into an improved version that will significantly increase th
```

This approach:

1. **Expertise Parameters**:

- Specific background ("15+ years of experience")
- Relevant domains ("HR, recruiting, and career coaching")

- Breadth of knowledge ("across multiple industries")

2. **Guiding Principles**:

- Numbered list of improvement strategies
- Gives priorities (achievements over duties)
- Specific techniques (quantification, action verbs)

3. **Success Criteria**:

- Clear goal ("increase the candidate's chances of getting interviews")
- Quality benchmarks ("impactful, professional, and compelling")
- Ethical boundaries ("while maintaining truthfulness")

4. **Creates Mental Framework**:

- Builds a conceptual structure for the task
- Establishes evaluation criteria for the output
- Serves as a checklist for the improvement process

This detailed context improves output quality by giving the model a better understanding of what's needed from the task. This is basically functioning as a short but effective training session on CV improvement.

8.3.1.3 Structured I/O Templates

```
// From pages/api/grade-cv.js - Structured output template
const prompt = `
Analyze the following CV thoroughly and provide a detailed assessment:

${cv}

Provide your analysis in the following JSON format, with scores from 1-10 where higher is better

{
  "overallScore": 0,
  "sections": {
    "format": {
      "score": 0,
      "feedback": ""
    },
    "content": {
      "score": 0,
      "feedback": ""
    },
    "impact": {
      "score": 0,
      "feedback": ""
    },
    "relevance": {
      "score": 0,
```

This template based approach:

1. **Defines Exact Output Structure**:

- Gives a JSON template with all required fields
- Shows expected data types (scores as numbers, feedback as strings)
- Format constraints (scores from 1-10, specific sections)

2. **Complete Analysis**:

- Breaks assessment into specific categories (format, content, impact, etc.)
- Asks for balanced feedback (strengths, weaknesses, suggestions)
- Evaluation criteria is consistent

3. **Reliable Parsing**:

- Creates predictable response structures
- Simplifies frontend processing of responses
- Reduces errors in data extraction

4. **Sets Response Boundaries**:

- Stops the model from adding unnecessary sections

The structured template approach is reinforced by the response_format: { type: "json_object" } parameter, which ensures the model produces valid JSON conforming to the specified structure[2]. This combination creates consistent, predictable outputs that can be reliably processed by the application's frontend[2].

8.3.1.4 Multi-Step Prompt Chaining

```
// From pages/api/improve-cv.js - Multi-step prompt chain
// Step 1: Extract key facts to preserve
const extractionPrompt = `Extract only the key factual information from this CV that MUST be pres
1. Name and contact information
2. Employment history (companies, positions, dates)
3. Education details (institutions, degrees, dates)
4. Technical skills and certifications
5. Languages and proficiency levels

Format as a simple bullet list. Include ONLY factual information, no subjective assessments.

CV to analyze:
${originalCV}`;

const keyFacts = await getOpenAIResponse(extractionPrompt, 'gpt-3.5-turbo');
```

```
// Step 2: Identify improvement opportunities
const analysisPrompt = `Analyze this CV and identify the top 5 specific opportunities for improve
1. Focus on actionable issues only
2. Prioritize by impact on interview chances
3. Be specific about problems, not just general categories

CV to analyze:
${originalCV}`;

const improvementAreas = await getOpenAIResponse(analysisPrompt, 'gpt-3.5-turbo');

// Step 3: Generate improved CV
const improvementPrompt = `Improve this CV while preserving these key facts:

KEY FACTS TO PRESERVE:
${keyFacts}

IMPROVEMENT OPPORTUNITIES:
${improvementAreas}

ORIGINAL CV:
${originalCV}

Create an improved version that addresses the identified issues while maintaining all factual inf
```

This chained approach:

1. **Breaks Complex Tasks into Stages**:

- Separates the fact extraction from the improvement function
- Divides analysis and implementation
- Creates focused subtasks with clear purposes

2. **Builds Context**:

- Each step builds on information from previous steps
- Later prompts include insights from earlier stages
- Creates a progression from analysis to implementation

3. **Implements Guardrails**:

- Explicitly preserves key factual information
- Prevents hallucination or fabrication

- Makes sure improvements focus on presentation, not content

4. **Optimises Model Usage**:

- Uses simpler models for fact extraction and analysis
- Reserves advanced models for complex improvement task(mentioned earlier in report)
- Balances cost and quality across the workflow

Multi step prompt chains lets the application tackle complex tasks like CV improvement in a more reliable way while still keeping the quality. This is more effective than using a single prompt, and gives more control over the process and output.

8.3.1.5 Explicit Constraints and Guardrails

The application uses clear constraints to prevent common AI issues:

```
JOB DESCRIPTION:
${jobDescription}

ADDITIONAL NOTES FROM APPLICANT:
${additionalNotes || "No additional notes provided."}

IMPORTANT CONSTRAINTS - YOU MUST FOLLOW THESE:
1. DO NOT invent or fabricate any experience, skills, or qualifications not mentioned in the CV
2. DO NOT mention specific company names unless they appear in the CV
3. DO NOT exceed 400 words total
4. DO NOT use clichéd opening lines like "I am writing to apply for..."
5. DO NOT include generic phrases that could apply to any job
6. DO NOT add any placeholder text like [Company Name] that needs to be filled in later

FORMAT REQUIREMENTS:
- Include proper business letter formatting with date and greeting
- Address to "Hiring Manager" if no specific name is provided
- Create 3-4 concise paragraphs with clear purpose
- Include a confident closing paragraph
- Sign with the applicant's full name

The cover letter must be specifically tailored to this particular job, highlighting the most rele
```

This:

1. **Prevents Common AI Problems**:

- Stops hallucination ("DO NOT invent or fabricate")

- Avoids placeholder text that would require the user to edit it
- Stops the AI from generating generic, unhelpful content

2. **Sets Clear Boundaries**:

- Sets specific word count limits, format requirements, and what information can and cannot be used

3. **Highlights Critical Requirements**:

- Uses distinctive formatting for constraints (capitalisation, numbered list)
- Puts constraints in a separate, labelled section

4. **Addresses Known Model Weaknesses**:

- Explicitly doesn't allow common AI writing patterns
- Prevents generic content
- Makes sure the output is professional and original

Explicit constraints are really important for features like cover letter generation, where the model might fall back on generic templates or fabricate details to make the letter seem more personalised.

8.3.1.6 Temperature Settings

The AI Job Helper uses different temperature settings based on task requirements:

```
// For factual analysis (low temperature)
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4o',
    messages: [
      // ... messages ...
    ],
    temperature: 0.3, // Low temperature for factual analysis
    // ... other parameters ...
  })
});
```

```
// For creative content (moderate temperature)
const response = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: 'gpt-4o',
    messages: [
      // ... messages ...
    ],
    temperature: 0.7, // Higher temperature for creative content
    // ... other parameters ...
  })
});
```

1. **Matches Temperature to Task Type**:

- Lower temperature (0.2-0.3) for analytical tasks (CV grading, fact extraction)
- Moderate temperature (0.5-0.7) for creative tasks (CV generation, cover letters)
- Higher temperature (0.7-0.8) for creative variations (multiple improvement suggestions)[2][4]

2. **Balances Creativity and Consistency**:

- Makes sure there's factual accuracy in analytical outputs
- Allows creative variety where needed
- Keeps a professional tone but still allows stylistic flexibility

3. **Adapts to User Expectations**:

- Gives precise, consistent analysis
- Also gives creative, unique content for generative features
- Matches output style to the feature's purpose

Temperature calibration is a subtle but crucial aspect of prompt engineering, shaping how the model interprets and responds to prompts.

8.3.2 Feature-Specific Prompt Engineering

8.3.2.1 CV Generation Prompts

```
// Template-specific instructions
let templateInstructions = '';
if (template === 'modern') {
  templateInstructions = 'Create a modern, clean CV with clear section headings, strategic use
} else if (template === 'traditional') {
  templateInstructions = 'Create a traditional, chronological CV with formal structure. Include
} else if (template === 'creative') {
  templateInstructions = 'Create a distinctive CV that balances creativity with professionalism
}

return `Please create a professional CV based on the following information:
```

```
FORMAT GUIDELINES:
1. Create clear, distinct sections with appropriate headings
2. Use bullet points for skills and achievements
3. Focus on accomplishments rather than responsibilities
4. Include all provided information organized in a professional structure
5. Format dates consistently throughout the document
6. Create a CV that would be competitive for job applications in the person's field

The CV should be ready for the applicant to use without needing further editing.`;
};
```

This code:

1. **Integrates Template Customisation**:

- Provides specific instructions based on the template the user selected

2. **Structures Input Data**:

- Organises information into sections
- Keeps all details the user provided
- Creates a data structure for the model to work with

3. **Provides Formatting Guidelines**:

- Outlines specific structural requirements
- Asks for focus on accomplishments
- Requests professional formatting

8.3.2.2 CV Improvement Prompts

```
const improvementPrompt = `
You are tasked with transforming the following CV into a significantly more effective and impactfu

Focus your improvements on these specific areas:
1. Replace generic job descriptions with specific, quantified achievements
2. Strengthen the impact of each bullet point with powerful action verbs
3. Add measurable results wherever possible (%, $, quantities, etc.)
4. Enhance the professional summary/objective to be more compelling
5. Improve overall structure, flow, and readability
6. Ensure consistent formatting and appropriate keyword usage
7. Eliminate clichés, redundancies, and vague language

CRITICAL REQUIREMENTS:
- Preserve all factual information (job titles, companies, dates, education)
- Do not invent accomplishments or experience not implied in the original
- Maintain approximately the same length (±10%)
- Keep the same overall structure and sections

Original CV to improve:
${originalCV}

Key facts that MUST be preserved:
${keyFacts}

Provide ONLY the improved CV as your response. Do not include explanations, notes, or commentary.
```

This code snippet:

1. **Targets Specific Improvement Areas**:

- Lists concrete improvement categories
- Focuses on common CV weaknesses
- Provides clear improvement direction

2. **Establishes Critical Boundaries**:

- Explicitly requires preserving factual information
- Stops the AI from making up fake achievements
- Sets constraints on length and structure

3. **Include Previous Analysis**:

- Includes key facts extracted in the processing step
- Emphasises critical information to keep

4. **Controls Output Format**:

- Requests only the improved CV, without commentary
- Creates a clean, document ready for the user immediately
- Gets rid of unnecessary explanation text

8.3.2.3 CV Grading Prompts

```
// From pages/api/grade-cv.js - Analytical grading prompt
const gradingPrompt = `
As an expert CV reviewer, conduct a comprehensive analysis of this CV:

${cv}

Your assessment should evaluate these key areas:
1. Visual Impact & Formatting (layout, readability, use of space)
2. Content Quality (relevance, depth, specificity)
3. Achievement Focus (results vs. responsibilities)
4. Language & Clarity (word choice, conciseness, impact)
5. Overall Effectiveness (competitive strength in job market)

For each area, provide:
- A score from 1-10 (where 10 is excellent)
- Specific evidence from the CV supporting your score
- Actionable recommendations for improvement

Also identify:
- Top 3 strengths with specific examples
- Top 3 weaknesses with improvement suggestions
- Overall competitiveness assessment

Format your response as a structured evaluation report with clearly labeled sections and a final
```

1. **Creates Framework**:

- Defines the assessment categories
- Creates a consistent scoring system
- Gives the structure for analysis

2. **Evidence Based Assessment**:

- Demands specific evidence from the CV
- Connects scores to examples
- Stops the AI doing vague evaluation

3. **Feedback**:

- Specific improvement recommendations are asked for
- Balances strengths and weaknesses
- Focuses on practical enhancements

4. **Professional Report Structure**:

- Gives a clear and professional format
- Organised, labelled sections
- Establishes professional tone

The grading prompt effectively turns the AI into a CV evaluation expert by giving it a clear framework for assessment, clear scoring criteria, and requirements for specific evidence and feedback.

8.3.2.4 Cover Letter Generation Prompts

```
// From pages/api/generate-cover-letter.js - Context-rich cover letter prompt
const coverLetterPrompt = `
Generate a compelling, tailored cover letter that connects the applicant's experience to the spec:

CV/RESUME:
${cv}

JOB DESCRIPTION:
${jobDescription}

APPLICANT NOTES:
${additionalNotes || "None provided"}

Create a cover letter that:
1. Opens with a strong, engaging first paragraph that shows genuine interest
2. Identifies 3-4 key qualifications from the CV that directly match the job requirements
3. Provides specific examples demonstrating these qualifications
4. Shows knowledge of the company/role based on the job description
5. Closes confidently with a clear call to action

FORMATTING:
```

```
FORMATTING:
- Use proper business letter format
- Address to "Hiring Manager" unless a name is specified in the job description
- Keep to 350-450 words total
- Include date, greeting, and professional sign-off
- Maintain a confident, enthusiastic but professional tone

ESSENTIAL GUIDELINES:
- Avoid clichés and generic statements that could apply to any job
- Do not fabricate experience or qualifications not mentioned in the CV
- Make specific connections between the applicant's background and job requirements
- Highlight transferable skills when direct experience is limited
- Emphasize the value the applicant would bring to the organization

The cover letter should sound natural, personalized, and convincing - not like a generic template
```

This approach:

1. **Makes use of Multiple Context Sources**:

- Uses both CV and job description(Fetched from FindWorkAPI)
- Gives a lot of context for personalisation

2. **Clear Structure**:

- Defines formatting

- Sets appropriate length parameters

3. **Focuses on Strategic Personalisation**:

- Requires that the model is matching qualifications to job requirements
- Says that it needs specific examples and connections
- Demands job/company specific content

4. **Stopping Common Problems**:

- Explicitly stops generic content
- Stops fabrication of experience
- Requires natural, language

The cover letter prompt uses multiple information sources (CV, job description) for generating this highly personalised cover letter. The detailed guidelines make sure the output is professionally formatted, job-specific, and compelling rather than generic or templated.

8.3.3 Business Impact of Effective Prompt Engineering

The prompt engineering in my project application delivers significant 'business' value and gives an edge over similar competitors:

1. **Output Quality and Consistency**:

- Produces professional quality documents consistently
- Maintains high standards
- Creates user ready outputs that need minimal editing

2. **Reduced AI Issues**:

- Less hallucinations and fabrications
- No generic or clichéd content
- Makes sure there's factual accuracy in any of the AI generated content

9 FindWork API

The FindWork API is an external job search service that I integrated into the project using their free API, to provide real time job listings that users can search for. This integration enhances the application beyond CV creation, turning it into more of a career tool that helps users not only create professional CVs but also find relevant job opportunities and generate tailored cover letters for those positions. FindWork.dev is a specialised job board API that focuses on providing structured job listing data, particularly for technology and developer roles. In the AI Job Helper application, it serves as the data source for the job search functionality, allowing users to discover employment opportunities that match their skills and preferences[2].

9.1 Integration Architecture

```
// From pages/api/search-jobs.js - FindWork API integration
const API_URL = "https://findwork.dev/api/jobs/";

export default async function handler(req, res) {
  // Get search parameters from request
  const { query, location } = req.query;

  // Get the API key from environment variables
  const API_KEY = process.env.FINDWORK_API_KEY;

  // Check if API key is available
  if (!API_KEY) {
    return res
      .status(500)
      .json({ error: "API key is missing. Please set it in .env.local." });
  }

  try {
    // Build search URL with parameters
    const url = new URL(API_URL);
    url.searchParams.append("search", query);
    url.searchParams.append("location", location);
```

```
    // Send request to FindWork API
    const response = await fetch(url, {
      method: "GET",
      headers: {
        Authorization: `Token ${API_KEY}`,
        "Content-Type": "application/json",
      },
    });

    if (!response.ok) {
      throw new Error(`Failed to fetch jobs: ${response.statusText}`);
    }

    // Parse and return results
    const data = await response.json();

    if (data && data.results) {
      res.status(200).json(data.results);
    } else {
      res.status(404).json({ message: "No jobs found for the given criteria." });
    }
  } catch (error) {
    console.error("Error fetching jobs from Findwork API:", error.message);
    res.status(500).json({ error: "Failed to fetch job data." });
  }
}
```

1. **Server Side**:

- API calls are made only using Next.js API routes on the server
- This makes a secure proxy between the frontend and the FindWork API
- Protects the API key from being exposed to users

2. **Environment Variable Security**:

- The API key is stored in environment variables (process.env.FINDWORK_API_KEY)
- Never exposed in client side code or browser accessible files

3. **Parameter Sanitisation**:

- User inputs (job title, location) are properly encoded in URL parameters
- Helps prevent injection attacks and makes sure there's proper URL formatting

- Creates well formed requests to the external API

4. **Error Handling**:

- try/catch block for handling API errors
- Appropriate HTTP status codes for different error conditions
- User friendly error messages that don't expose sensitive information

This server-side proxy approach is good for securely integrating with external APIs. It allows the application to hide sensitive credentials that shouldn't be exposed to users[2].

9.2 Frontend Integration

```
// From components/meetups/JobSearch.js - Frontend integration
const handleSearch = async () => {
  // Basic validation
  if (!jobTitle || !location) {
    alert("Please enter both job title and location.");
    return;
  }

  // Set loading state
  setLoading(true);
  setJobs([]);
  setCoverLetter("");
  setCoverLetterJobId(null);

  try {
    // Construct API URL with properly encoded parameters
    const apiUrl = `/api/search-jobs?query=${encodeURIComponent(
      jobTitle
    )}&location=${encodeURIComponent(location)}`;

    // Make request to our Next.js API route
    const response = await fetch(apiUrl);
```

```
    if (!response.ok) {
      throw new Error(`Failed to fetch jobs: ${response.statusText}`);
    }

    // Process and display results
    const data = await response.json();

    if (data && data.length > 0) {
      setJobs(data);
    } else {
      alert("No jobs found for the given criteria.");
      setJobs([]);
    }
  } catch (error) {
    console.error("Error fetching jobs:", error);
    alert("An error occurred while fetching job data. Please try again.");
  } finally {
    setLoading(false);
  }
};
```

1. **User Friendly**:

- Clear input fields for job title and location
- Search button with a loading indication
- Input validation to make sure necessary search parameters are input by the user

2. **API Integration**:

- Calls the Next.js API route (/api/search-jobs) rather than directly accessing FindWork
- Properly encodes URL parameters for security as mentioned before
- Handles loading states and errors

3. **Results Display**:

- Renders job listings
- Shows key information (job title, company, location)
- Includes links to original job postings so the user can apply

4. **State Management**:

- Uses React state to track search parameters, results, and loading status
- Clears previous results when starting a new search
- Gives correct feedback for empty results

5. **Error Handling**:

- Catches and processes API errors
- Displays user friendly error messages
- Properly resets UI state when errors occur

9.3 Integration with CV Features

```
try {
  // Get job description
  let fullJobDescription = job.description || job.text;

  // Make API call to generate cover letter
  const response = await fetch("/api/generate-cover-letter", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      cvId: selectedCVId,
      jobTitle: job.role || job.title,
      company: job.company_name || job.company,
      jobDescription: fullJobDescription ||
        `This is a ${job.role || job.title} position at ${job.company_name || job.company} loca
      location: job.location,
    }),
  });
```

1. **Workflow**:

- Users search for jobs using FindWork data
- Select a saved CV from their profile
- Generate a tailored cover letter for a specific job listing

2. **Combining data**:

- Job data from FindWork is combined with user's CV data
- Both datasets are sent to the cover letter generation endpoint
- Generates the content matching the user's qualifications to the job requirements

3. **Fallbacks for the Content Generation**:

- Uses available job description from FindWork when present
- Creates fallback description from basic job details when full description isn't available
- This makes sure the cover letter generation has enough context even if the data from the API is missing

4. **User Experience**:

- Each job listing includes its own "Generate Cover Letter" button
- Loading states and success/error messages are specific to each job
- Downloaded files are named according to the company the user is applying to

9.4 Security

```
// From pages/api/search-jobs.js - Security patterns
// URL parameter encoding for safe API requests
const url = new URL(API_URL);
url.searchParams.append("search", query);
url.searchParams.append("location", location);

// API key protection in server-side code
const response = await fetch(url, {
  method: "GET",
  headers: {
    Authorization: `Token ${API_KEY}`, // Never exposed to client
    "Content-Type": "application/json",
  },
});

// From components/meetups/JobSearch.js - Client-side security
// Parameter encoding in frontend
const apiUrl = `/api/search-jobs?query=${encodeURIComponent(
  jobTitle
)}&location=${encodeURIComponent(location)}`;
```

Security features:

1. **API Key Protection**:

- Keys stored in environment variables, never in source code

2. **Input Sanitization**:

- Uses encodeURIComponent() for user inputs in URL parameters and prevents injection attacks and URL manipulation.

3. **Next.js API Route Proxy**:

- Creates a security barrier between client and external API
- Controls what data is returned to the frontend

4. **Output Sanitisation**:

- Job descriptions are displayed using safe rendering techniques
- External links use rel="noopener noreferrer" for security
- Prevents potential XSS issues from untrusted content[2]

5. **Error Information Limitation**:

- Detailed error logs stored server-side only with generic error messages returned to users.
- Prevents information being leaked through error messages.

9.5 Benefit of FindWork API

1. **Career Solution**:

- Transforms the application from a CV builder to a full job application tool
- The CV creation to job search to cover letter generation workflow works seamlessly and is user friendly.

2. **Enhanced User Experience**:

- Users can search for relevant jobs directly within the application
- Cover letters can be tailored to specific job listings, far quicker than if you edited a cover letter template yourself.

3. **User Engagement**:

- Gives users reasons to return to the application beyond CV creation
- Increases user retention

4. **Competitive Differentiation**:

- Elevates the application beyond basic CV template tools
- Offers integrated functionality that many competitors lack

- Creates unique value in the career tools market

10 Users (Authentication and Session Management)

The AI Job Helper implements a user authentication and session management system that provides secure user identification, access control, and login. This system lets users create accounts, log in securely, access their saved CVs, and log out.

The implementation follows modern best practices, with password hashing, session management with encrypted cookies, and proper separation of concerns between frontend and backend components.

10.1 Core Technical Components

10.1.1 User Data

```
// From models/User.js - User Schema Definition
const UserSchema = new mongoose.Schema({
  // --- Email Field --- //
  email: {
    type: String,
    required: [true, 'Please provide an email address.'],
    unique: true,
    trim: true,
  },
  // --- Password Field --- //
  password: {
    type: String,
    required: [true, 'Please provide a password.'],
    minlength: [6, 'Password must be at least 6 characters long'],
  },
  // --- Name Field --- //
  name: {
    type: String,
    required: [true, 'Please provide your name.'],
    trim: true
  },
  // --- CreatedAt Field --- //
  createdAt: {
    type: Date,
    default: Date.now
  },
```

Features of the User model:

1. **User Attributes**:

- email: Used as the unique identifier for login, with validation requirements
- password: Stored as a hashed value (processed with bcrypt before saving)
- name: User's full name
- createdAt: Timestamp for account creation

2. **Data Validation**:

- Required fields prevent missing user data
- Unique emails makes sure there's only one account per email address
- Password minimum length (6 characters) for basic security standards
- Trim modifier removes the whitespace from text fields

3. **MongoDB Integration**:

- Schema defines the structure for the "users" collection in MongoDB
- Uses Mongoose's model system for structured database operations
- Has hot-reloading protection (mongoose.models.User || mongoose.model(...))

10.1.2 Session Management with Iron Session

```
// From lib/session.js - Session Configuration
import { withIronSession } from "next-iron-session";

// --- Session Configuration Options --- //
// This options object configures how the session cookie behaves.
const sessionOptions = {
  // Password for encrypting the cookie data (must be at least 32 chars)
  password: process.env.SECRET_COOKIE_PASSWORD,
  // Name of the cookie
  cookieName: "cv_builder_session",
  // Cookie options
  cookieOptions: {
    // Secure ensures cookie is sent only over HTTPS (except in development)
    secure: process.env.NODE_ENV === "production",
    // HttpOnly prevents JavaScript from accessing the cookie
    httpOnly: true,
    // SameSite helps prevent CSRF attacks
    sameSite: "strict",
    // Sets cookie expiration time (2 weeks in seconds)
    maxAge: 60 * 60 * 24 * 14, // 14 days
  },
};
```

```
// Higher-Order Functions for adding session capabilities to API routes and pages
export function withSessionRoute(handler) {
  return withIronSession(handler, sessionOptions);
}

export function withSessionPage(handler) {
  return withIronSession(handler, sessionOptions);
}

// Helper function to access user data from session
export async function getUserFromSession(req) {
  try {
    const user = req.session.get("user");
    if (!user) {
      return null;
    }
    return user;
  } catch (error) {
    console.error("Error getting user from session:", error);
    return null;
  }
}
```

1. **Stateless Session Architecture**:

- Uses encrypted cookies rather than server side session storage
- All session data is stored client side but encrypted for security
- Works with the serverless architecture of Next.js

2. **Session Utilities**:

- withSessionRoute adds session handling to API routes
- withSessionPage adds session handling to server side page rendering
- getUserFromSession provides access to user data in a standardised way

This stateless session way of approaching this provides security while keeping good performance and scalability, which is fit for modern web apps.

10.1.3 Security

10.1.3.1 Password Hashing

```
// From pages/api/auth/signup.js - Password hashing
// Generate a salt and hash the plain text password using bcrypt.
// 12 is the number of salt rounds - a good balance between security and performance.
const hashedPassword = await bcrypt.hash(password, 12);

// From pages/api/auth/login.js - Secure password verification
// Compare the provided password with the stored hash
const isPasswordValid = await bcrypt.compare(password, user.password);
```

- Uses the bcrypt library for password hashing
- Uses asynchronous functions for performance
- Never stores or transmits plaintext passwords
- Uses the secure compare function to verify passwords without direct comparison

10.1.3.2 Encrypted Session Cookies

```
// From lib/session.js - Secure cookie configuration
const sessionOptions = {
  // Password for encrypting the cookie data
  password: process.env.SECRET_COOKIE_PASSWORD,
  // Name of the cookie
  cookieName: "cv_builder_session",
  // Cookie options
  cookieOptions: {
    // Secure ensures cookie is sent only over HTTPS (except in development)
    secure: process.env.NODE_ENV === "production",
    // HttpOnly prevents JavaScript from accessing the cookie
    httpOnly: true,
    // SameSite helps prevent CSRF attacks
    sameSite: "strict",
    // Sets cookie expiration time
    maxAge: 60 * 60 * 24 * 14, // 14 days
  },
};
```

- Encrypts all session data using a strong password

- Restricts cookie access to HTTPS connections in production
- Prevents JavaScript access to the cookie (XSS protection)
- Implements SameSite policy to stop CSRF
- Sets expiration time to limit session duration

10.1.3.3 Authorization for Protected Resources

```
// From pages/api/cvs.js - Protecting user resources
async function handler(req, res) {
  // Connect to the database
  await dbConnect();

  // Get the logged in user
  const user = await getUserFromSession(req);

  // Check if user is authenticated
  if (!user) {
    return res.status(401).json({ success: false, error: 'Not authenticated' });
  }

  switch (method) {
    case 'GET':
      try {
        // If an ID is provided, get a specific CV
        if (req.query.id) {
          const cv = await CV.findOne({
            _id: req.query.id,
            userId: user.id // Security: Only return CV if it belongs to the logged in user
          });
          // ...
        }
        // Otherwise, get all CVs for the current user
        else {
```

- Verifies authentication status before allowing access to protected resources
- Ensures users can only access their own data (userId filtering)

11 MongoDB

I have mentioned mongo and mongoose throughout, here I'll add some extra points not mentioned.

```
// From lib/mongodb.js - Database connection configuration
// Retrieve the MongoDB connection string from environment variables
const MONGODB_URI = process.env.MONGODB_URI;

// Validate that the connection string is available
if (!MONGODB_URI) {
  throw new Error(
    'FATAL ERROR: Please define the MONGODB_URI environment variable inside .env.local (for devel
  );
}

// Global connection caching to optimize serverless environment performance
let cached = global.mongoose;

if (!cached) {
  cached = global.mongoose = { conn: null, promise: null };
}

async function dbConnect() {
  // Return existing connection if available
  if (cached.conn) {
    return cached.conn;
  }
```

```
  // Create new connection if none exists or is in progress
  if (!cached.promise) {
    console.log('Establishing new database connection...');
    const opts = {
      serverSelectionTimeoutMS: 5000,
      socketTimeoutMS: 45000,
    };

    cached.promise = mongoose.connect(MONGODB_URI, opts)
      .then((mongooseInstance) => {
        console.log("MongoDB connection established successfully.");
        return mongooseInstance;
      })
      .catch((error) => {
        console.error("MongoDB connection error:", error);
        cached.promise = null;
        throw error;
      });
  }

  // Wait for connection to resolve
  try {
```

Key MongoDB implementation features:

1. **Environment-Based Configuration**:

- Connection string stored in environment variables
- Early validation to stop runtime errors

2. **Connection Optimisation**:

- Global caching mechanism to reuse connections
- Needed for serverless environments like Next.js API routes
- Prevents creating new connections for each serverless function call

3. **Performance Tuning**:

- Custom connection options for timeouts and socket behavior
- Proper error handling

Nested CV Schema

```
// From models/CV.js - CV data model with nested schemas
// Nested schema for experience entries
const ExperienceSchema = new mongoose.Schema({
  title: String,
  company: String,
  startDate: String,
  endDate: String,
  responsibilities: String,
});

// Nested schema for education entries
const EducationSchema = new mongoose.Schema({
  degree: String,
  institution: String,
  graduationDate: String,
});
```

1. **Nested Schema Structure:**

- Array fields for multiple entries (education, experience)
- Maintains structured data

2. **Document Relationships**:

- References between documents via ObjectId
- Proper document ownership with user references

12 Problem Solving

In this section I'll go ever some of the problems I faced during the development of the project and how I dealt with them. I mentioned a lot of the direct problems I fixed earlier in the specific sections, so in this section I'll mention other ones or go into more detail on the previously mentioned ones.

One of the challenges for this project was with the generated text from the ChatGPT API. A problem with a lot of AI generated content is the fabrication of information or the AI making things up that you didn't ask for. Another problem I had was that the first CV's that I was generating, we're low quality and didn't follow a lot of good CV practices. This meant the user would be better off not using my application. Another related problem was that I felt my initial project wasn't very useful, and there wasn't much of a reason for using my project over just putting the CV into ChatGPT. The solution to all of the problems I just mentioned was learning prompt engineering. Learning how to craft a long and detailed prompt to battle these problems was challenging but once I had enough practice and research done, I could get better at creating these prompts. This added the proper value that I needed for this project.

13 Ethics

AI generation in and AI in general in the job market present a lot of ethical challenges that have to be considered. With hiring agencies using more and more AI tools to automatically filter out applicants, it seems like people continue to have to jump through more and more hoops in order to keep up. Candidates who could be incredibly suited to the job they applied for, could be instantly rejected by an AI because their CV doesn't include specific ATS keywords. That's what gave me the initial idea for this project, as so many new graduates find it hard to keep up with the current job market, and might not know what they're doing wrong when they keep getting rejected.

Another ethical question is with applicants and candidates using AI to fabricate their skills on their CV to make them appear more qualified then they are. My idea with this project was for it to be used as a tool, and not teach users how to lie or trick companies to get interviews, but teach them how they can expand on what skills they actually have and how they can come across more professional. A lot of the things that are necessary for the job application process are not taught to students, so they are left to fend for themselves. The AI generation used in this project is to enhance what is real about the applicant, so that it stays authentic to them.

14 Conclusion

The AI Job Helper has evolved from simple concepts to a functioning application that blends deep job market research with modern coding practices and technologies for web applications. Working on the project was very personal for me as it's topic covers something that I was very relevant for me and many other soon to be graduates that are trying to start in their career.

What starter as a technical challenge has become a practical tool that I intend to use in my personal life moving forward. I really wanted to remove the fear that people new to the job market have and help them make up CVs without the intimidation of starting from a blank page. This is why I had a focus on allot of UI and UX features so that it was as user friendly as possible. I felt it was unfair for people to be expected to have a lot of prior knowledge that's not necessary to their job. I didn't want someone like a software engineer, get rejected from their dream job that they're a perfect match for, get rejected because they had poor CV writing skills or didn't know a companies ATS keywords. This project let me apply my knowledge and research so that people can be given a fair chance in the modern job market.

Overall I found the project to be a success and I was delighted with how the final product came out. I reached all my main goals that I wanted to, and got to work with all the technologies that I wanted to learn more about. The research and development of the project has given me incredibly useful skills that I can carry with me for the rest of my career.

15 References

[1] Forbes 8 Tips To Optimize Your Resume For Applicant Tracking Systems, [Online]. Available: https://www.forbes.com/sites/lucianapaulise/2024/08/29/optimize-resume-for-applicant-tracking-systems/

[2] ChatGPT [Online]. Available: ChatGPT.com

[3] Next.js "Next.js by Vercel" [Online]. Available: https://nextjs.org/

[4] OpenAi API Reference. [Online]. Available: https://platform.openai.com/docs/api-reference/introduction